

Assignment Briefing Sheet (2025/26 Academic Year)

Section A: Assignment title, important dates and weighting

Assignment title:	Ref/Def Coursework	Group or individual:	Individual
Module title:	Programming for Software Engineers	Module code:	7COM1025
Module leader:	Hui Cheng	Moderator's initials:	YZ
Submission deadline:	June 2026	Target date for return of marked assignment:	July 2026

You are expected to spend about 40 hours to complete this assignment to a satisfactory standard.

This assignment is worth 50% of the overall assessment for this module.

Section B: Student(s) to complete

Student ID number	Year Code

Notes for students

- For undergraduate modules, a score above 40% represent a pass performance at honours level.
- For postgraduate modules, a score of 50% or above represents a pass mark.
- Late submission of any item of coursework for each day or part thereof (or for hard copy submission only, working day or part thereof) for up to five days after the published deadline, coursework relating to modules at Levels 0, 4, 5, 6 submitted late (including deferred coursework, but with the exception of referred coursework), will have the numeric grade reduced by 10 grade points until or unless the numeric grade reaches or is 40. Where the numeric grade awarded for the assessment is less than 40, no lateness penalty will be applied.
- Late submission of referred coursework will automatically be awarded a grade of zero (0).
- Coursework (including deferred coursework) submitted later than five days (five working days in the case of hard copy submission) after the published deadline will be awarded a grade of zero (0).
- Regulations governing assessment offences including Plagiarism and Collusion are available from <https://www.herts.ac.uk/about-us/governance/university-policies-and-regulations-uprs/uprs> (please refer to UPR AS14)
- Guidance on avoiding plagiarism can be found here: https://herts.instructure.com/courses/61421/pages/referencing-avoiding-plagiarism?module_item_id=779436
- Modules may have several components of assessment and may require a pass in all elements. For further details, please consult the relevant Module Handbook (available on Studynet/Canvas, under Module Information) or ask the Module Leader.

Assignment Briefing Sheet (2025/26 Academic Year)

This Assignment assesses the following module Learning Outcomes (from Definitive Module Document):

- LO4 - Design and implement a substantial software artefact.
- LO5 - Evaluate, choose, and employ appropriate data structures and algorithms.
- LO6 - Refactor part of a software item to achieve a specified goal.
- LO7 - Be able to design a test suite and perform a set of tests.

Assignment Brief:

See specification provided on Canvas

Submission Requirements:

Students should submit on Studynet the following three items:

1. a single pdf document containing the report with the UML class diagrams and version control snapshots,
2. a zipped project folder containing the source codes folder, test folder, and an executable jar file for running the final program,
3. a screen recording video showing the functionalities of your final program.

Marks awarded for:

Source code will be checked against the report and the video to confirm both that the work has been implemented by yourself, and its design.

- . UML Class diagram. (10 marks)
- . System functions. (40 marks)
- . Use of version control, with suitable commit messages. (5 marks)
- . Appropriate design and implementation. (30 marks)
- . Use of JUnit testing. (10 marks)
- . Report format and writing quality. (5 marks)

Total: 100 marks

Type of Feedback to be given for this assignment:

A detailed mark sheet will be provided; verbal feedback after the marks is released.

Implement a system according to the following description.

The Excel Academy Tuition Centre (EATC) needs software for managing lesson bookings made by the students. The centre offers different group tuition lessons on both Saturday and Sunday. The tuition lesson subjects are English, Math, Verbal Reasoning, and Non-verbal Reasoning. Each lesson can accommodate 4 students at most.

Each student has a record stored on the system holding their name, gender, DoB, address, and emergency contact phone number. For either day (Saturday or Sunday), there are 2 tuition lessons per day: 1 in the morning, and 1 in the afternoon. The lesson price is different for different subjects. The lesson price for the same subject will remain the same even if they run at a different time.

A student wishing to book a lesson needs to select a specific time from the tuition timetable. Students are allowed to change the date/time of the booked lessons, provided spaces are still available for the newly selected date/time in the same subject. A student is not allowed to change the booking from one subject to another different subject, however, cancelling a booking is allowed. A student can book as many lessons as they want so long as there is no time conflict.

On attending a lesson, a student will first be checked in (i.e., changing the lesson status from “booked” to “attended” for this student). After each tuition lesson, students can write a review of the lesson they have attended and provide a numerical rating of the lesson ranging from 1 to 5 (1: Very dissatisfied, 2: Dissatisfied, 3: Ok, 4: Satisfied, 5: Very Satisfied). The review and rating information will be recorded in the system.

After four weeks (four weekends), the system can print:

- a report containing the number of students per tuition lesson, along with the average rating;
- a report containing the tuition lesson subject which has generated the highest income.

In your final program, set up at least 4 different tuition lesson subjects and 10 students. You need to design the lesson prices and at least 8 weekends of tuition timetable by yourself.

Note: you do not need any kind of external database for this program. The final program should be self-contained. The reports can be printed to System.out, or output to a suitable GUI interface. You also do not need any kind of security protocol.

Your tasks:

1. Create a class diagram, highlighting the main Java classes in the system and their associations. Include the attributes and key methods.
2. Implement the system, using JUnit tests to confirm the functionality. Record all the functionalities of your final system in a video.
3. Write a short report (no page limit, about 8 pages should be sufficient) explaining your design and implementation.

Discuss:

- a) Any assumptions made about the system, in addition to the above description.
- b) The overall structure and design of your system.
- c) The JUnit tests.
- d) Any refactoring used during the development of the system.
- e) Any design patterns/design principles used.

Deliverables:

- A. Develop your code using any version control software. Take snapshots of your project repository and commit messages. Include the snapshots in your report.
- B. Submit to Studynet:
 1. Your report with the UML class diagrams and version control snapshots in a single PDF document
 2. A zipped project folder containing the source codes folder, test folder, and an executable jar file for running the final program.
 3. A screen recording video showing the functionalities of your final program.